

Chapter 4: Automatic Differentiation

One concept, one exercise per slide — practice as you go

Concept: Why Automatic Differentiation?

- Numerical differentiation (finite differences) needs one extra forward pass PER parameter
- Symbolic differentiation produces expressions that explode in size with composition depth
- Automatic differentiation computes exact gradients for ALL parameters in about one extra pass
- AD is neither purely numerical nor purely symbolic — it applies the chain rule mechanically to a recorded computation



Exercise: A model has 7 billion parameters. Roughly how many forward passes would numerical differentiation need to compute the full gradient? Why is this completely impractical?

Concept: The Computational Graph

- Every computation is a Directed Acyclic Graph (DAG) of elementary operations
- Leaf nodes: inputs and parameters. Root node: the final scalar output (the loss)
- Each node stores its forward value AND, after backward(), its gradient
- Each node also stores a local backward rule — the chain-rule recipe for its own operation

 **Exercise: For $f = (a + b) * c$, draw the computation as a graph: how many nodes are there, and which are leaves?**

Concept: The Multivariable Chain Rule (Summation Form)

- If a node v feeds into several downstream nodes $w_1...w_k$, its gradient is a SUM:
- $\partial L / \partial v = \sum_i (\partial L / \partial w_i) \cdot (\partial w_i / \partial v)$
- This single equation is the entire mathematical content of backpropagation
- The sum is why gradient contributions must be ACCUMULATED, not overwritten



Exercise: A variable x is used twice in $f = x * x$ (i.e., x feeds two multiplication inputs of the same node). Using the summation form, explain in words why df/dx is NOT simply the derivative of one usage alone.

Concept: Local Backward Rules

- Every elementary operation (add, multiply, exp, ...) has a simple, known local derivative
- Addition: gradient passes through unchanged to both inputs
- Multiplication: each input's gradient is (incoming gradient) $\times$ (the OTHER input's value)
- Complex functions are just long chains of these simple rules



Exercise: For $out = a * b$, if the incoming gradient at 'out' is 5, and $a=3$, $b=4$, what are the gradients that flow back to a and to b ?

Concept: Topological Sort — The Ordering Guarantee

- The backward pass must visit every node AFTER all of its downstream consumers
- This ordering is exactly the reverse of a topological sort of the forward graph
- Without correct ordering, a node's gradient could be read before it's fully accumulated
- Implemented via recursive depth-first search with a visited set



Exercise: In the graph $a \rightarrow c$, $b \rightarrow c$, $c \rightarrow d$, why must d's gradient be fully computed before c's backward rule runs, and why must c's gradient be fully computed before a's or b's backward rule runs?

Concept: Forward Mode (JVP) vs. Reverse Mode (VJP)

- Forward mode: one pass computes the gradient with respect to ONE input — needs N passes for N inputs
- Reverse mode: one pass computes the gradient with respect to ALL inputs at once — needs M passes for M outputs
- Neural network training: N (parameters) is huge, M (the scalar loss) is 1
- Decision rule: use reverse mode when $N \gg M$, forward mode when $M \gg N$



Exercise: A function has 3 inputs and 1,000,000 outputs. Which mode (forward or reverse) is cheaper, and why?

Concept: The Reverse Accumulation Algorithm

- 1. Run the forward pass, recording every operation and its inputs
- 2. Topologically sort the resulting graph
- 3. Initialize the root node's gradient to 1 ($\partial L / \partial L = 1$)
- 4. Walk the sorted nodes in REVERSE order, applying each local backward rule



Exercise: Why does the algorithm initialize the root's gradient to exactly 1, rather than 0 or the loss value itself?

Concept: Gradient Accumulation at Shared Nodes

- When a node is consumed by multiple downstream operations, EVERY backward closure must use +=
- Using = instead of += silently discards all but the last contribution
- This is a silent bug: no exception, no NaN — just a mathematically wrong gradient
- Directly required by the summation form of the chain rule from earlier in this deck



Exercise: A buggy implementation writes `self.grad = other.data * out.grad` instead of `self.grad += other.data * out.grad` inside a multiply node's backward closure. For $f = x * x$, what wrong gradient value would this produce at $x=3$ (the correct answer is 6)?

Concept: `zero_grad` and Gradient Accumulation Across Steps

- Gradients accumulate (`+=`) WITHIN one `backward()` call — that's required, not a bug
- But gradients also persist ACROSS multiple `backward()` calls unless explicitly reset
- Forgetting to zero gradients between training steps silently sums gradients across steps
- This is exactly what `optimizer.zero_grad()` exists to prevent



Exercise: You call `backward()` twice in a row on two different batches without calling `zero_grad()` in between. What happens to the parameter gradients, and how would this show up as a training symptom?

Concept: Complexity of a Backward Pass

- Forward pass: $O(|V|)$ — one visit per node, each doing $O(c)$ work
- Backward pass: $O(|E|)$ — work per node is proportional to its number of downstream consumers
- For most graphs, $|E| = O(|V|)$, so a full backward pass costs about the same as one forward pass
- This is why reverse-mode AD is 'free' relative to computing the loss itself



Exercise: A network's forward pass takes 10 milliseconds. Roughly how long should you expect the backward pass to take, and why?

Concept: Gradient Checkpointing — Trading Time for Memory

- Naively, EVERY node's forward value must stay in memory until backward() consumes it
- Peak memory scales as $O(\text{depth})$ for a deep network — this can exceed available memory
- Checkpointing stores only every $\sqrt{L}$ -th activation, recomputing the rest during backward()
- Optimal trade-off: memory drops to $O(\sqrt{L})$ at the cost of one extra forward pass



Exercise: A 100-layer network stores every activation naively, needing memory for 100 layers. Using the $\sqrt{L}$ checkpointing strategy, roughly how many activations need to be held in memory at once instead?

Mastery Check: Are You Ready to Move On?

If you can answer every question below confidently, you have mastered Chapter 4.

1. Explain why automatic differentiation is neither purely numerical nor purely symbolic differentiation.
2. Draw the computational graph for a small expression and identify its leaves, interior nodes, and root.
3. State the multivariable chain rule in summation form and explain why it requires gradient accumulation.
4. Derive the local backward rule for multiplication and for addition.
5. Explain why the backward pass must run in reverse topological order.
6. Explain the difference between forward-mode (JVP) and reverse-mode (VJP) autodiff, and when to use each.
7. Walk through the reverse accumulation algorithm's four steps from memory.
8. Explain why using '=' instead of '+=' in a backward closure produces a silent, wrong gradient.
9. Explain the difference between within-call gradient accumulation and across-step accumulation, and why `zero_grad()` is needed.
10. State the time complexity of one backward pass and explain why it does not scale with the number of parameters directly.
11. Explain the gradient checkpointing trade-off and derive why $\sqrt{L}$ is the optimal checkpoint spacing.
12. Implement a minimal scalar autograd engine (Value class) supporting `+`, `*`, and `backward()`.